constructs as well as modularity constructs. It also aims to familiarize students with the concept of common libraries and frameworks, including those to facilitate API-based access to resources.

2. **SDF-Data-Structures**: Fundamental Data Structures – This knowledge unit aims to develop core concepts relating to Data Structures and associated operations. Students should understand the important data structures available in the programming language or as libraries, and how to use them effectively, including choosing appropriate data structures while designing solutions for a given problem.
3. **SDF-Algorithms**: Algorithms – This knowledge unit aims to develop the foundations of algorithms and their analysis. The KU should also empower students in selecting suitable algorithms for building modest-complexity applications.
4. **SDF-Practices**: Software Development Practices – This knowledge unit develops the core concepts relating to modern software development practices. It aims to develop student understanding and basic competencies in program testing, enhancing the readability of programs, and using modern methods and tools including some general-purpose IDE.
5. **SDF-SEP**: Society, Ethics, and the Profession – This knowledge unit aims to develop an initial understanding of some of the ethical issues related to programming, professional values programmers need to have, and the responsibility to society that programmers have. This knowledge unit is a part of the SEP Knowledge Area.

Core Hours

Knowledge Unit	CS Core	KA Core
Fundamental Programming Concepts and Practices	20	
Fundamental Data Structures	6 + 6 (AL)	
Algorithms	3 + 3 (AL)	
Software Development Practices	5	
Society, Ethics and the Profession	Included in SEP hours	
Total	**43**	

Note: The CS Core hours include 9 hours shared with AL, but counted here.

Knowledge Units

SDF-Fundamentals: Fundamental Programming Concepts and Practices

CS Core:

1. Basic concepts such as variables, primitive data types, expressions, and their evaluation

2. How imperative programs work: state and state transitions on execution of statements, flow of control
3. Basic constructs such as assignment statements, conditional and iterative statements, basic I/O
4. Key modularity constructs such as functions (and methods and classes, if supported in the language) and related concepts like parameter passing, scope, abstraction, data encapsulation (See also: FPL-OOP)
5. Input and output using files and APIs
6. Structured data types available in the chosen programming language like sequences (e.g., arrays, lists), associative containers (e.g., dictionaries, maps), others (e.g., sets, tuples) and when and how to use them (See also: AL-Foundational)
7. Libraries and frameworks provided by the language (when/where applicable)
8. Recursion
9. Dealing with runtime errors in programs (e.g., exception handling).
10. Basic concepts of programming errors, testing, and debugging (See also: SE-Construction, SEC-Coding)
11. Documenting/commenting code at the program and module level.(See also: SE-Construction)
12. Develop a security mindset. (See also: SEC-Foundations)

Illustrative Learning Outcomes:

CS Core:

In these learning outcomes, the term "develop" means "design, write, test, and debug."

1. Develop programs that use the fundamental programming constructs: assignment and expressions, basic I/O, conditional and iterative statements.
2. Develop programs using functions with parameter passing.
3. Develop programs that effectively use the different structured data types provided in the language like arrays/lists, dictionaries, and sets.
4. Develop programs that use file I/O to provide data persistence across multiple executions.
5. Develop programs that use language-provided libraries and frameworks (where applicable).
6. Develop programs that use APIs to access or update data (e.g., from the web).
7. Develop programs that create simple classes and instantiate objects of those classes (if supported by the language).
8. Explain the concept of recursion and identify when and how to use it effectively.
9. Develop recursive functions.
10. Develop programs that can handle runtime errors.
11. Read a given program and explain what it does.
12. Write comments for a program or a module specifying what it does.
13. Trace the flow of control during the execution of a program.
14. Use appropriate terminology to identify elements of a program (e.g., identifier, operator, operand).

SDF-Data-Structures: Fundamental Data Structures

CS Core: **(**See also: AL-Foundational**)**

1. Standard abstract data types such as lists, stacks, queues, sets, and maps/dictionaries, including operations on them.

2. Selecting and using appropriate data structures.
3. Performance implications of choice of data structure(s).
4. Strings and string processing.

Illustrative Learning Outcomes:

CS Core:

1. Write programs that use each of the key abstract data types provided in the language (e.g., arrays, tuples/records/structs, lists, stacks, queues, and associative data types like sets, dictionaries/maps).
2. Select the appropriate data structure for a given problem.
3. Explain how the performance of a program may change when using different data structures or operations.
4. Write programs that work with text by using string processing capabilities provided by the language.

SDF-Algorithms: Algorithms

CS Core: **(**See also: AL-Foundational**,** AL-Complexity**)**

1. Concept of algorithm and notion of algorithm efficiency
2. Some common algorithms (e.g., sorting, searching, tree traversal, graph traversal)
3. Impact of algorithms on time-space efficiency of programs

Illustrative Learning Outcomes:

CS Core:

1. Explain the role of algorithms for writing programs.
2. Demonstrate how a problem may be solved by different algorithms, each with different properties.
3. Explain some common algorithms (e.g., sorting, searching, tree traversal, graph traversal).
4. Explain the impact on space/time performance of some algorithms.

SDF-Practices: Software Development Practices

CS Core: (See also: SE-Construction)

1. Basic testing, including test case design
2. Use of a general-purpose IDE, including its debugger
3. Programming style that improves readability
4. Specifying functionality of a module in a natural language.

Illustrative Learning Outcomes:

CS Core:

1. Develop tests for modules and apply a variety of strategies to design test cases.
2. Explain some limitations of testing programs.
3. Build, execute, and debug programs using a modern IDE and associated tools such as visual debuggers.

4. Apply basic programming style guidelines to aid readability of programs such as comments, indentation, proper naming of variables, etc.
5. Write specifications of a module as module comment describing its functionality.

SDF-SEP: Society, Ethics, and the Profession

CS Core:

1. Intellectual property rights of programmers for programs they develop.
2. Plagiarism and academic integrity.
3. Responsibility and liability of programmers regarding code they develop for solutions. (See also: SEC-Foundations)
4. Basic professional work ethics of programmers.

Illustrative Learning Outcomes:

CS Core:

1. Explain/understand some of the intellectual property issues relating to programs.
2. Explain/understand when code developed by others can be used and proper ways of disclosing their use.
3. Explain/understand the responsibility of programmers when developing code for an overall solution (which may be developed by a team).
4. Explain/understand one or more codes of conduct applicable to programmers.

Professional Dispositions

- **Self-Directed:** Students must seek out solutions to issues on their own (e.g., using technical forums, FAQs, discussions). Resolving issues is an important part of becoming proficient in programming.
- **Experimental:** Students must experiment with language features to understand them and to quickly prototype solutions. This helps in learning about programming language features.
- **Technical curiosity:** Students must develop interest in understanding how programs are executed, how programs and data are stored in memory, etc. This will help build better mental models of the underlying execution system on which programs run.
- **Adaptable:** Students must be willing to learn and use different tools and technologies that facilitate software development. Tools are commonly used while programming and new tools often emerge – using tools effectively and learning the use of new tools will help.
- **Persistent:** Students must continue efforts until, for example, a bug is identified, a program is made robust and handles all situations, etc. This will help as programming requires effort and ability to persevere till a program works satisfactorily.
- **Meticulous:** Students must pay attention to detail and use orderly processes while programming. The underlying machine is unforgiving and there is no room for even small errors in the programs as they can cause major failures.